Sub-Graphs

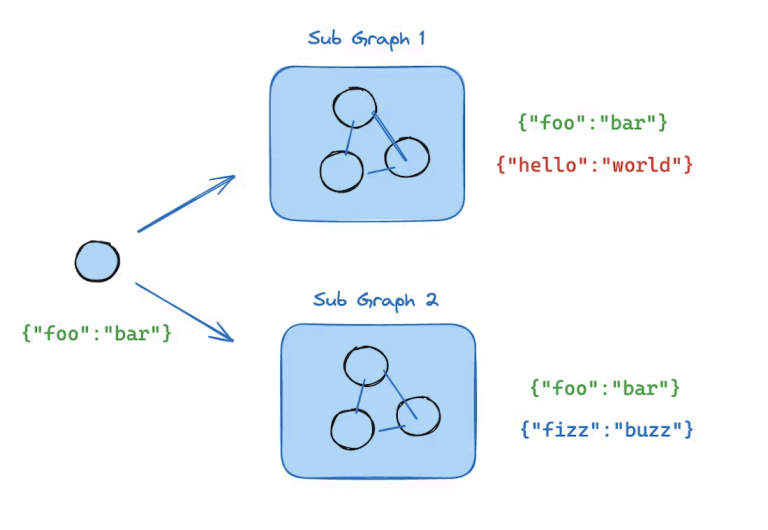
Sub-Graphs

A subgraph is a graph that is used as in another graph. They are useful for:

- Building multi-agent systems
- Re-using a set of nodes in multiple graphs
- Distributing development: when you want different teams to work on different parts of the graph independently

Implementation 1: Sub-Graph as Node

- For this, the parent graph and subgraph can communicate over a shared state key (channel) in the schema



▼ **Demo**

1. Compile the subgraph
2. Add it as a node

```
from typing_extensions import TypedDict
from langgraph.graph.state import StateGraph, START

class State(TypedDict):
    foo: str

# Subgraph

def subgraph_node_1(state: State):
    return {"foo": "hi! " + state["foo"]}

subgraph_builder = StateGraph(State)
subgraph_builder.add_node(subgraph_node_1)
```

```
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph = subgraph_builder.compile()

# Parent graph

builder = StateGraph(State)
builder.add_node("node_1", subgraph)
builder.add_edge(START, "node_1")
graph = builder.compile()
```

Implementation 2: Sub-Graph within Node

- If we want our sub-graphs to have different schema keys, we have to define a node thatt invokes the sub-graph

▼ **Demo**

```
from typing_extensions import TypedDict
from langgraph.graph.state import StateGraph, START

class SubgraphState(TypedDict):
    bar: str

# Subgraph

def subgraph_node_1(state: SubgraphState):
    return {"bar": "hi! " + state["bar"]}

subgraph_builder = StateGraph(SubgraphState)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph = subgraph_builder.compile()

# Parent graph

class State(TypedDict):
    foo: str

def call_subgraph(state: State):
    # Transform the state to the subgraph state
    subgraph_output = subgraph.invoke({"bar": state["foo"]})
    # Transform response back to the parent state
    return {"foo": subgraph_output["bar"]}

builder = StateGraph(State)
builder.add_node("node_1", call_subgraph)
```

```
builder.add_edge(START, "node_1")
graph = builder.compile()
```

Other Details

- **State tracking: y**ou can inspect the graph state via `graph.get_state(config)`. To view the subgraph state, you can use `graph.get_state(config, subgraphs=True)`.
- **Streaming: t**o include outputs from subgraphs in the streamed outputs, you can set the subgraphs option in the stream method of the parent graph. This will stream outputs from both the parent graph and any subgraphs.

▼ **Streaming Sub-Graph Demo**

```
for chunk in graph.stream(
    {"foo": "foo"},
    subgraphs=True,
    stream_mode="updates",
):
    print(chunk)
```

- You only need to **provide the checkpointer when compiling the parent graph**. LangGraph will automatically propagate the checkpointer to the child subgraphs.
- If you want the subgraph to **have its own memory**, you can compile it with the appropriate checkpointer option. This is useful in **multi-agent** systems, if you want agents to keep track of their internal message histories

▼ **Seperate Checkpointer Demo**

```
subgraph_builder = StateGraph(...)
subgraph = subgraph_builder.compile(checkpointer
=True)
```